Machine Learning 2024 Fall Final Project

Team Name: ML(B)project

Team Member: 羅瑋宸、王晉任

1. Introduction

1.1 Background

Predicting baseball game outcomes is a challenging and multifaceted problem, requiring an understanding of the underlying game dynamics, player performances, and contextual factors. This project utilizes machine learning (ML) to predict the outcomes of baseball games using historical game data provided in two stages. The focus is on binary classification to determine if the home team will win or lose.

1.2 Data Description

The dataset includes historical game data from January to July (training) and August to December (testing) for the years 2016 to 2023. For the second stage, the task is to predict outcomes for all games in 2024. The features include contextual information, recent performance metrics, and seasonal statistics, ..., as detailed in table below.

Feature Type
Game Informamtion
Team and Pitcher Rest
Peacher Information
Recent Performance (10-Game Rolling Average)
Team Seasonal Statistics
Team Seasonal Batting
Team Seasonal Pitching
Pitcher Seasonal Performance

1.3 Problem framing

The task is a binary classification problem:
Our objective is to minimize classification error using multiple ML approaches.

1.4 Performance Evaluation

Let $w_{i,t}$ be the ground-truth of win-or-lose indicator of home-team i and data $\hat{w}_{i,t}$ be our predicted result (same format as $w_{i,t}$. The point-wise error is defined as the 0/1 error of $w_{i,t}$ as below:
$err(\hat{w}_{i,t}, w_{i,t}) = [\hat{w}_{i,t} \neq w_{i,t}]$
Then, for any set, we can calculate its average err over all examples.

2. Method

2.1 Preprocess

2.1.1 Handling Missing Values

Missing values were imputed with the column mean for numerical features. Categorical features (e.g., is_night_game) were mapped to binary values.

2.1.2 Feature Engineering

We don't do any particular feature engineering, we take all features into consideration.

2.1.3 Normalization

We use sklearn's normalization to do the max normalization which is to normalize all the data in the same column by the max value. The description can be translated as the following equation:
$\tilde{x}_{i,d} = \frac{x_{i,d}}{max(|x_{1,d}|,|x_{2,d}|,...,|x_{n,d}|)}$, where $\tilde{x}_{i,d}$ is the new feature in the i-th row, d-th dimension and $x_{i,d}$ is the original feature in the i-th row, d-th dimenstion.

2.2 Models

2.2.1 Logistic Regression

Logistic Regression is a statistical model used for binary classification. It predicts the probability of a binary outcome (0 or 1) using a sigmoid function to map the output of a linear model to a range between 0 and 1.

2.2.2 LightGBM

LightGBM is a gradient-boosting framework that utilizes tree-based learning algorithms to achieve high efficiency and performance. It incorporates two key techniques to optimize the gradient-boosting decision tree (GBDT) process:

- Gradient-Based One-Side Sampling (GOSS)
- Exclusive Feature Bundling (EFB)

The boosting mechanism in LightGBM follows the traditional iterative gradient-boosting decision tree process:

- The first tree learns to fit the target variable.
- Subsequent trees are trained to minimize the residuals (errors) between the current model's predictions and the ground truth.
- Gradients of the errors are propagated through the system to guide the training of each tree.

The iteration process of gradient boosting can be represented mathematically as: $F_{i+1} = F_i - f_i$, where F_i is the strong model at step i and f_i is the weak model (tree) added at step i.

2.2.3 Random Forest

Random Forest is an ensemble method that creates multiple decision trees, each trained on a random subset of the data using bootstrapping. At each split in a tree, it selects a random subset of features to reduce correlation between trees. For classification tasks, the final prediction is determined by majority voting, while for regression tasks, it averages the predictions of all trees. This randomness helps improve generalization and reduces the risk of overfitting. The algorithm is widely used in machine learning because of its versatility and effectiveness across a wide range of datasets.

2.2.4 Catboost

CatBoost is a gradient boosting algorithm specifically designed to handle categorical features efficiently while maintaining high performance for both classification and regression tasks.

CatBoost stands for "Categorical Boosting" and includes unique techniques like:

- Automatic Handling of Categorical Features: It converts categorical features into numerical representations using an innovative encoding method that avoids overfitting.
- Ordered Boosting: Instead of using the entire dataset to compute gradients, it processes data sequentially, reducing target leakage and ensuring unbiased training.
- Efficient Training: It uses optimized implementations to speed up training while preserving accuracy.

CatBoost is known for its ability to work out of the box with minimal parameter tuning, making it user-friendly and highly effective on datasets with many categorical variables.

2.3 Parameter

2.3.1 Logistic regression

- max_iter=1000: The maximum number of iterations for the optimization algorithm to converge.
- penalty: l2
- Regularization Strength: 1.0
- solver: lbfgs

2.3.2 LightGBM

We use Optuna, an optimization framework, to perform hyperparameter tuning for a LightGBM model and perform 5-fold cross-validation on the dataset. The result:

- n_estimators=420: The number of boosting iterations or trees to be built in the model.
- learning_rate=0.011597715995336785: The step size at each iteration while moving toward a minimum of the loss function.
- max_depth=11: The maximum depth of each tree.
- num_leaves=113: The maximum number of leaves in one tree.
- min_child_samples=5: The minimum number of data points (samples) required to create a leaf.
- colsample_bytree=0.5559735264016111: The fraction of features to be randomly sampled for each tree.
- subsample=0.8842576112526516: The fraction of samples to be used for fitting individual base learners (trees).

2.3.3 Random Forest

We use Optuna, an optimization framework, to perform hyperparameter tuning for a Random Forest model and perform 5-fold cross-validation on the dataset. The result:

- n_estimators=300: The number of trees in the forest.
- max_depth=5: The maximum depth of each tree in the forest.
- min_samples_split=3: The minimum number of samples required to split an internal node.

2.3.4 Catboost

- iterations=500: The number of boosting iterations or trees to build.
- learning_rate=0.05: The step size at each iteration for the updates.
- depth=10: The maximum depth of the trees.
- verbose=0: Controls the amount of information printed during training.

2.4 Experimental Setting

- Programming Language: Python 3.11.8
- OS: macOS

3. Model Comparision

3.1 Accuracy

We evaluate it by the Kaggle stage score. The higher score means higher accuracy.

	1. Logistic Regression	2. LightGBM	3. Random Forest	4. Catboost
Public score (stage1)	0.56262	0.57230	0.58263	0.56617
Private score (stage1)	0.57499	0.56851	0.57402	0.56041
Public score (stage2)	0.57132	0.55897	0.58887	0.55232
Private score (stage2)	0.54738	0.51715	0.55310	0.52941

Accuracy comparison:

- public test (stage1): 3 > 2 > 4 > 1
- private test (stage1): 1 > 3 > 2 > 4
- public test (stage2): 3 > 1 > 2 > 4
- private test (stage2): 3 > 1 > 4 > 2

3.2 Stability

Consistency of performance across stages. We evaluate it by $S = \mid (Stage2\ score) - (Stage1\ score) \mid$. The smaller S means the higher stability.

	1. Logistic Regression	2. LightGBM	3. Random Forest	4. Catboost
S (Public)	0.0087	0.0133	0.00624	0.01385
S (Private)	0.02761	0.05136	0.02092	0.031

Stability comparison:

- Public test: 3 > 1 > 2 > 4
- Private test: 3 > 1 > 4 > 2

3.3 Efficiency

We evaluate it by the time usage. The smaller time usage means the higher efficiency.

	1. Logistic Regression	2. LightGBM	3. Random Forest	4. Catboost
Efficiency	High	High	Medium	Medium

3.4 Scalability

Performance with increasing data size. We evaluate it by feeding different size of training data and calculate its E_{out}. Increasing the data size with higer E_{out} means worse scalability.

	1. Logistic Regression	2. LightGBM	3. Random Forest	4. Catboost
Scalability	Medium	High	Medium	Medium

3.5 Interpretability

Clarity of model behavior and feature contributions.

	1. Logistic Regression	2. LightGBM	3. Random Forest	4. Catboost
Interpretability	High	Medium	High	Medium

4. Final Recommandation

We choose Random Forest as our final recommandation.

- Pros:
  - Best Accuracy: Achieves the highest scores across both public and private tests in Stage 2 and also performs well in Stage1.
  - Stability: Performs the best for both test scenarios.
  - Interpretability: Provides feature importance metrics for deeper insights.
- Cons:
  - Efficiency: Training time is higher compared to LightGBM.
  - Scalability: Moderate performance with increasing data size.

Random Forest emerges as the best approach for this project due to its superior accuracy and stability across public and private test scenarios. Although LightGBM remains a strong

contender for its efficiency, Random Forest provides a better balance of accuracy and stability and interpretability, and thus we recommend it for baseball game outcome prediction. We think future work could explore ensemble techniques combining Random Forest and LightGBM for even better performance.

5. Work Distribution

	Work Load
羅瑋宸	Implemnt Logistic, LightGBM, Random Forest, Catboost algotihm and write the report.
王晉任	Implement Random Forest 10-fold, Random Forest 20-fold, logistic with loocv and some content in report.

6. Reference

[1] https://www.geeksforgeeks.org/how-to-normalize-data-using-scikit-learn-in-python/
[2] https://www.ibm.com/think/topics/random-forest
[3] https://www.avanwyk.com/an-overview-of-lightgbm/
[4] https://www.datacamp.com/tutorial/catboost